

OWASP Top 10 for LLM Applications (2026)

Course handout — Securing AI: Northwind Outfitters

This handout lists all ten risks in current 2026 OWASP ranking order, with the generic description, how each one showed up today in CampaignBot, InsightAgent, or the guided-risk discussion, and the mitigation to take away.

| LLM01 — Prompt Injection

Description: Crafted input — direct or indirect — manipulates the model into ignoring its original instructions, so untrusted text is treated as a command.

In this course: CampaignBot ① a marketer's brief overrode the system prompt directly. CampaignBot ② a planted note in a customer record achieved the same result with no one touching the brief field at all. InsightAgent ①② showed the same two shapes again, but now driving a tool-using planner instead of just text.

Mitigation: Separate untrusted data from instructions structurally — don't rely on "ignore previous instructions" rules written into the system prompt. Pattern-matching validators (as shown with `ENABLE_VALIDATORS=true`) catch the demo payload, not the class of attack — treat them as a speed bump, not a fix.

| LLM02 — Sensitive Information Disclosure

Description: The system reveals data — personal, proprietary, or otherwise sensitive — that it shouldn't, often through outputs or retained context rather than a direct attack.

In this course: InsightAgent's happy-path read query pulled customer PII into working memory even though only a summary was requested — the data was there to be exposed long before any attack scenario ran.

Mitigation: Scope what enters working memory to only what the task actually needs. Minimise and redact PII in context rather than assuming it's safe because no attack is in play.

| LLM03 — Excessive Agency

Description: The system is granted more autonomy — permissions, tool access, ability to act — than its task requires, so a manipulated or mistaken decision has real-world consequences.

In this course: InsightAgent ④ exported a CSV and ⑤ sent a campaign with no human approval step and no permission check in the tool layer (`permission_check: false` in source) — the model could act, not just answer.

Mitigation: Scope tool permissions to the specific action type, not blanket access. Add a human-in-the-loop approval gate on irreversible actions — a design pattern to build in from the start, not a patch bolted on after.

| LLM04 — Supply Chain

Description: Third-party models, datasets, or plugins are trusted without verification, and a compromise anywhere in that chain becomes a compromise of the system built on top of it.

In this course: Covered as a guided discussion — a purchased or fine-tuned model, or a dataset from an external vendor, is a trust boundary whether or not you wrote the training code yourself.

Mitigation: Treat model and dataset dependencies like code dependencies: maintain an AI-BOM (AI bill of materials) and know the provenance of anything you didn't create in-house before it ships.

LLM05 — Data and Model Poisoning

Description: Tampered training data teaches the model a hidden, harmful pattern that standard evaluation does not catch, because the model still performs well on every metric being measured.

In this course: The poisoning notebook walkthrough showed a model scoring 100% on a standard evaluation while a backdoor — a brand name that always predicted positive sentiment because it always had in the training data — fired reliably on a trigger input.

Mitigation: Data provenance matters before training begins, not after. Run label-consistency checks on any third-party dataset — the kind of check demonstrated in the notebook is something you can run on any dataset today, not just a teaching example.

LLM06 — Unbounded Consumption

Description: The system allows resource-intensive operations — long agentic loops, unrestricted API calls — without limits, opening the door to denial of service or runaway cost.

In this course: Covered as a guided risk — a scripted, tight-loop call against a “summarise my segments” feature with no rate limit or per-tenant cost ceiling turned a single customer’s script into an unusual spike on the API bill.

Mitigation: Rate limit per user or tenant, put step limits on agentic loops, and treat cost alerting as a business control, not only a security one — for a SaaS product this is a margin risk as much as an abuse risk.

LLM07 — Misinformation

Description: The model generates false but fluent and convincing content — invented facts, fabricated citations — with no signal to the reader that anything is wrong.

In this course: Covered as a guided risk — a summary citing a specific, invented industry benchmark figure got forwarded to a client without anyone checking, because nothing about the output read as uncertain.

Mitigation: Ground claims in retrievable sources and require citations. Put human review in front of any AI-generated figure before it ships externally — confidence in the output is not evidence it’s correct.

LLM08 — Hidden Context Exposure

(Renamed from “System Prompt Leakage” in the 2026 revision; broadened scope — covers any hidden context surfacing, not only the system prompt itself.)

Description: Internal instructions, business logic, or other context the system was never meant to show ends up visible to the user.

In this course: InsightAgent © — the planner’s own debug output printed the system prompt when asked to, and the “hidden” policy or reasoning behind a decision was sitting in plain text the whole time.

Mitigation: Don’t rely on the system prompt as a hiding place for secrets or business logic delegates shouldn’t see. Assume any hidden context can surface, and design so nothing sensitive depends on it staying hidden.

LLM09 — Vector and Embedding Weaknesses

Description: A RAG retrieval layer treats anything in its vector store as legitimate context, including documents nobody vetted — so a poisoned entry gets retrieved and used exactly like a trusted one.

In this course: Covered as a guided risk, illustrated with the Northwind refund-policy scenario — an attacker-planted document, worded closely enough to a real policy to be retrieved for “What’s our refund policy?”, got incorporated into the model’s context and reflected in the response. The model didn’t hallucinate; it followed instructions — the wrong ones.

Mitigation: Treat ingestion as untrusted input. Enforce tenant and source isolation at the retrieval layer, and attach provenance metadata to every document so retrieval can distinguish vetted content from anything bulk-uploaded by a third party.

| LLM10 — Improper Output Handling

Description: The model’s output is rendered or executed downstream without validation, so a competently generated but malicious payload — HTML, a command, a query — does exactly what it was written to do.

In this course: CampaignBot ③ — an onerror payload reached the browser via innerHTML and fired. The model wasn’t “hacked”; it was simply a competent HTML generator feeding an app that rendered its output unsanitised.

Mitigation: Encode output before it’s rendered. Use `textContent`, not `innerHTML`, and apply a Content Security Policy as a second line of defence.

| Cross-cutting pattern

Across every risk on this list, the same structural fix recurs in different clothes: **separate untrusted input from trusted instructions, scope permissions and context to the minimum the task needs, and put a human or a hard limit in front of anything irreversible or costly**. No single prompt change fixes any of these — they are architecture and process decisions, made before the feature ships.